

Table des matières

Rapport - Labo sans fil	1
Introduction	1
Manipulations et observations	1
Visualisation TTN et identification des gateways LoRaWAN	1
Test de simulation des données	2
Test et optimisation d'un décodeur JavaScript TTN	2
Accès au conteneur ALPHANET-DS	3
Intégration Webhook HTTP dans ALPHANET-DS	3
Affichage des données	5
Gestion des alertes	6
Webhook JSON vs Protocol Buffers	6
Conclusion	6
Annexes	8

Rapport - Labo sans fil

Introduction

Dans ce laboratoire, nous nous sommes intéressés à la transmission et au traitement de données dans un contexte IoT à l'aide du réseau LoRaWAN et de la plateforme TTN (The Things Network).

L'objectif était de comprendre le fonctionnement complet d'une chaîne de communication, depuis l'envoi de données par un capteur jusqu'à leur réception, leur décodage et leur exploitation sur un serveur. Nous avons notamment observé les messages uplink, mis en place un décodeur de payload, puis configuré un webhook HTTP afin de recevoir les données dans un conteneur.

Nous avons ensuite développé un script PHP permettant de traiter, stocker et afficher les données reçues, ainsi que d'identifier des situations anormales. Enfin, nous avons comparé différents formats de transmission (JSON et Protocol Buffers) afin d'évaluer leur impact sur l'exploitation des données.

Ce laboratoire s'inscrit dans la continuité des notions vues en réseau et en développement web, et permet d'illustrer concrètement le fonctionnement d'une architecture IoT complète, depuis la communication sans fil jusqu'au traitement côté serveur.

Manipulations et observations

Visualisation TTN et identification des gateways LoRaWAN

Après l'authentification sur le site TTN, nous sommes arrivés sur le dashboard de notre capteur LoRaWAN. Nous avons sélectionné l'onglet "Live data" du dashboard, puis appuyé sur le bouton du capteur afin de visualiser notre première donnée transmise.

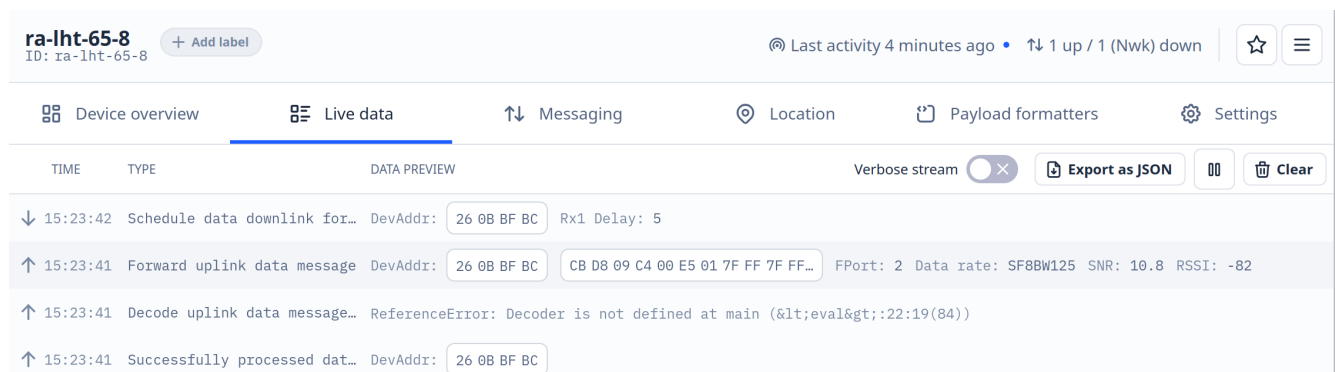


Figure 1: Première donnée transmise

Nous observons les étapes de traitement de l'uplink: 1. Forward uplink: réception du message 2. Decode uplink: tentative de décodage (échoue, pas de décodeur) 3. Schedule downlink: éventuelle réponse planifiée 4. Successfully processed: fin du traitement

En cliquant sur une de ces étapes, cela ouvre le détail de cette dernière au format JSON. C'est ici que nous pouvons voir par quel gateway notre message passe.

The screenshot shows the TTN dashboard interface. At the top, the device ID 'ra-lht-65-8' is displayed. Below the navigation bar, the 'Live data' tab is selected. A table of events is shown, with the 'Decode uplink data message' event selected. The 'Event details' panel on the right shows the following JSON data:

```

{
  "firm_payload": "y9gJxADlAX//f/8=",
  "rx_metadata": [
    {
      "gateway_ids": {
        "gateway_id": "hearc-ing-tinf-ca2",
        "eui": "313332323C006B00"
      },
      "timestamp": 1930315115,
      "rx_timestamp": 1930315115
    }
  ]
}

```

Figure 2: Gateway dans le détail de l'étape

Dans notre cas, nous pouvons lire que notre message passe par un gateway de la HE-Arc.

Test de simulation des données

Dans l'onglet "Messaging", puis la section "Simulated uplink" du dashboard, nous pouvons simuler un message qui pourrait être envoyé par le capteur. C'est pratique quand l'appareil physique n'est pas à portée de main.

The screenshot shows the 'Simulate uplink' form in the TTN dashboard. The 'FPort' field is set to 1. The 'Payload' field contains the hexadecimal string 'CC 54 08 0D 01 78 01 7F FF 7F FF'. Below the form is a blue 'Simulate uplink' button. A green success message box at the bottom right indicates 'Success: Uplink sent'.

Figure 3: Exemple de simulation d'uplink

Ensuite, nous pouvons vérifier dans l'onglet "Live data" que nous recevons bien un message, comme s'il provenait du capteur.

Test et optimisation d'un décodeur JavaScript TTN

Nous avons ensuite mis en place un décodeur afin de "décrypter" les messages en hexadécimal. Nous sommes repartis de l'exemple fourni du cours.

Avec cette base, les messages en hexadécimal deviennent un format JSON qui nous retourne des informations sur la batterie:

ra-lht-65-8 ID: ra-lht-65-8 + Add label Last activity 7 minutes ago • ↑ 2 up / 1 (Nwk) down ☆ ☰

Test

Byte payload CC 54 08 0D 01 78 01 7F FF 7F FF **FPort** 1 **Test decoder**

Decoded test payload

```
{
  "data": {
    "battery_status": 3,
    "battery_voltage": 3156
  }
}
```

Figure 4: Résultats du décodeur de base

Puis nous l'avons complété en suivant le commentaire en haut du code.

Lien vers le code complet: <code/payload-decode.js>

Avec le décodeur complété nous pouvons maintenant voir en plus les informations du capteur (humidité et température):

Test

Byte payload CC 54 08 0D 01 78 01 7F FF 7F FF **FPort** 1 **Test decoder**

Decoded test payload

```
{
  "data": {
    "battery_status": 3,
    "battery_voltage": 3156,
    "humidity": 37.6,
    "temp1": 20.61
  }
}
```

Figure 5: Résultats du décodeur complété

Accès au conteneur ALPHANET-DS

Après la connexion à la machine distante via SSH, nous avons mis à jour le système du conteneur, installé netcat et ouvert un serveur TCP sur le port 80 grâce à la commande `nc`.

Ensuite, depuis un navigateur web, nous sommes allés sur l'adresse <http://193.72.186.153:8087/script.php>. Nous voyons bien que le serveur ouvert précédemment a bien reçu une requête GET avec les entêtes client (User-Agent, Host, Accept, etc).

Intégration Webhook HTTP dans ALPHANET-DS

Installation de PHP et déploiement de l'application Nous avons installé PHP en mode standalone. Nous avons ensuite créé un répertoire `/html` afin d'héberger les fichiers de l'application. Nous avons ensuite lancé le serveur

```

user-8@ds-tinf-ra-sf-8:~$ sudo nc -l -p 80
GET /script.php HTTP/1.1
Host: 193.72.186.153:8087
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:148.0) Gecko/20100101 Firefox/148.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
DNT: 1
Sec-GPC: 1
Priority: u=0, i

```

Figure 6: Serveur TCP ouverture et réception

web intégré de PHP pour servir le contenu du répertoire courant.

Ce serveur minimal permet de tester rapidement une application.

Un script PHP de base nous a été fourni, nous l'avons déployé sur le conteneur à l'aide d'une commande scp.

Afin de simuler le comportement d'une future intégration avec TTN, nous avons testé notre script PHP en envoyant une requête HTTP POST.

```

📁 sans-fil/code on 📁 master [?] via 📁 v25.8.1 via 📁 v8.3.6
● > echo '{ "a": "b"}' | POST -E http://193.72.186.153:8087/script.php
POST http://193.72.186.153:8087/script.php
User-Agent: lwp-request/6.76 libwww-perl/6.76
Content-Length: 12
Content-Type: application/x-www-form-urlencoded

200 OK
Connection: close
Date: Thu, 23 Apr 2026 15:06:25 GMT
Host: 193.72.186.153:8087
Content-Type: text/html; charset=UTF-8
Client-Date: Thu, 23 Apr 2026 15:06:25 GMT
Client-Peer: 193.72.186.153:8087
Client-Response-Num: 1
X-Powered-By: PHP/8.2.30

```

Figure 7: Test serveur php minimal

Intégration Webhook avec TTN Sur le site TTN, nous avons créé une nouvelle intégration webhook qui renseigne le lien de notre script PHP.

En allant via un navigateur sur notre script, nous pouvons voir ce message dans notre serveur php, ce qui nous intéresse c'est le payload:

Le payload ci-dessus reçu n'était pas exploitable directement. Nous avons toutefois déjà effectué des tests avec un payload formatter. Nous l'avons activé.

Maintenant, les données deviennent interprétables et peuvent être lues au format JSON.

```
[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo3-a-8"},"dev_eui":"A840414881827810","join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["as:up:01KQEJKJ3XYC809TE7ZM39T664"],"rpc:/ttn.lorawan.v3.AppAs/SimulateUplink:45c68805-blee-44f0-bffe-e26acf286879"],"received_at":"2026-04-30T06:53:16.285394516Z","uplink_message":{"f_port":1,"frm_payload":"zFQIDQF4AX//f/8=","rx_metadata":[{"gateway_ids":{"gateway_id":"test"},"rssi":42,"channel_rssi":42,"snr":4.2}],"settings":{"data_rate":{"lora":{"bandwidth":125000,"spreading_factor":7}},"frequency":"868000000"},"simulated":true}
[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 [200]: POST /script.php
[Thu Apr 30 08:53:16 2026] 34.255.49.188:21672 Closing
```

Figure 8: Test webhook without payload formatter

```
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo3-a-8"},"dev_eui":"A840414881827810","join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["as:up:01KQEN1PV7VFEBWAJ0919ZQV21"],"rpc:/ttn.lorawan.v3.AppAs/SimulateUplink:fc94507f-221a-413e-a9ef-6fcffc080a07"],"received_at":"2026-04-30T07:35:57.030807453Z","uplink_message":{"f_port":1,"frm_payload":"zFQIDQF4AX//f/8=","decoded_payload":{"data":{"battery_status":3,"battery_voltage":3156,"humidity":37.6,"temp1":20.61}},"rx_metadata":[{"gateway_ids":{"gateway_id":"test"},"rssi":42,"channel_rssi":42,"snr":4.2}],"settings":{"data_rate":{"lora":{"bandwidth":125000,"spreading_factor":7}},"frequency":"868000000"},"simulated":true}
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 [200]: POST /script.php
[Thu Apr 30 09:35:57 2026] 34.255.49.188:34066 Closing
```

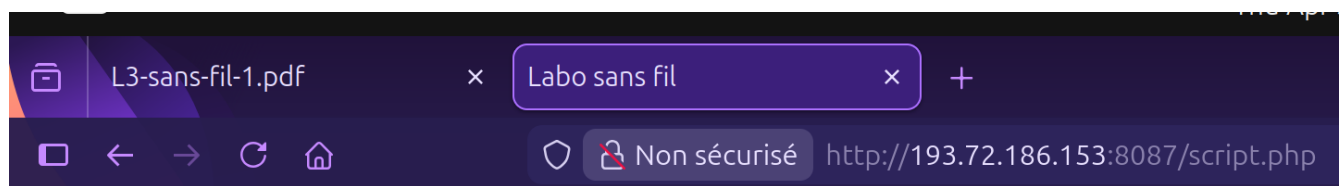
Figure 9: Test webhook with payload formatter

Affichage des données

Après la mise en place du webhook et du déploiement du script PHP sur le conteneur, nous avons mis en place l'affichage des données reçues.

Le script récupère les messages envoyés par TTN via une requête HTTP POST. Les données sont extraites depuis le payload JSON et stockées dans un fichier local afin de conserver la dernière valeur reçue.

Ces informations sont ensuite affichées sur une page web générée en PHP. Les valeurs de température, d'humidité et de batterie sont directement visibles dans le navigateur.



Données capteurs

Température: 10.9 °C

Humidité: 38.1 %

Batterie: 3049 mV

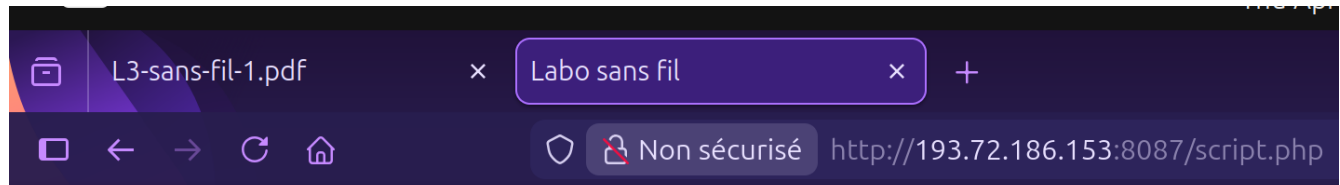
Figure 10: Affichage des données

La page se met à jour automatiquement toutes les 10 secondes afin de permettre un suivi quasi temps réel des données du capteur.

Gestion des alertes

En complément de l’affichage des données brutes, nous avons ajouté un système d’alertes afin d’interpréter les valeurs reçues et détecter des situations anormales.

Le script PHP récupère les données capteurs (température, humidité et tension de batterie) et les compare à des seuils définis. Si une valeur sort de la plage considérée comme normale, une alerte correspondante est générée.



Données capteurs

Température: 20.61 °C

Humidité: 37.6 %

Batterie: 3156 mV

Alertes

- Batterie faible (3156 mV)

Figure 11: Gestion des alertes

Les alertes générées sont ensuite affichées dynamiquement sur la page web sous forme de liste. Cela permet une lecture rapide de l’état du système sans analyser directement les valeurs numériques.

Webhook JSON vs Protocol Buffers

Afin d’obtenir des logs exploitables, nous sommes revenus au script PHP de base et avons désactivé le payload formater.

Nous avons ensuite simulé un uplink depuis TTN afin d’observer la requête POST reçue par notre serveur. Dans un premier temps, nous avons utilisé le format JSON (format par défaut), puis nous avons testé le format Protocol Buffers.

En format JSON, les données sont lisibles et structurées. Le payload est présent dans le champ `frm_payload`. Bien que ce contenu ne soit pas directement interprétable, il reste facilement décodable et exploitable.

En revanche, en format Protocol Buffers, les données reçues apparaissent sous forme binaire. Elles ne sont pas lisibles dans le terminal et ne peuvent pas être traitées directement par le script PHP.

Nous en déduisons que le format JSON est plus adapté dans notre cas, car il permet une manipulation simple des données, contrairement au format Protocol Buffers qui nécessite un décodage spécifique. Cependant, ce dernier est plus compact et optimisé pour la transmission, ce qui le rend plus efficace côté machine.

Conclusion

Au cours de ce laboratoire, nous avons observé le fonctionnement d’un capteur LoRaWAN via TTN, notamment la réception des messages uplink et leur traitement.

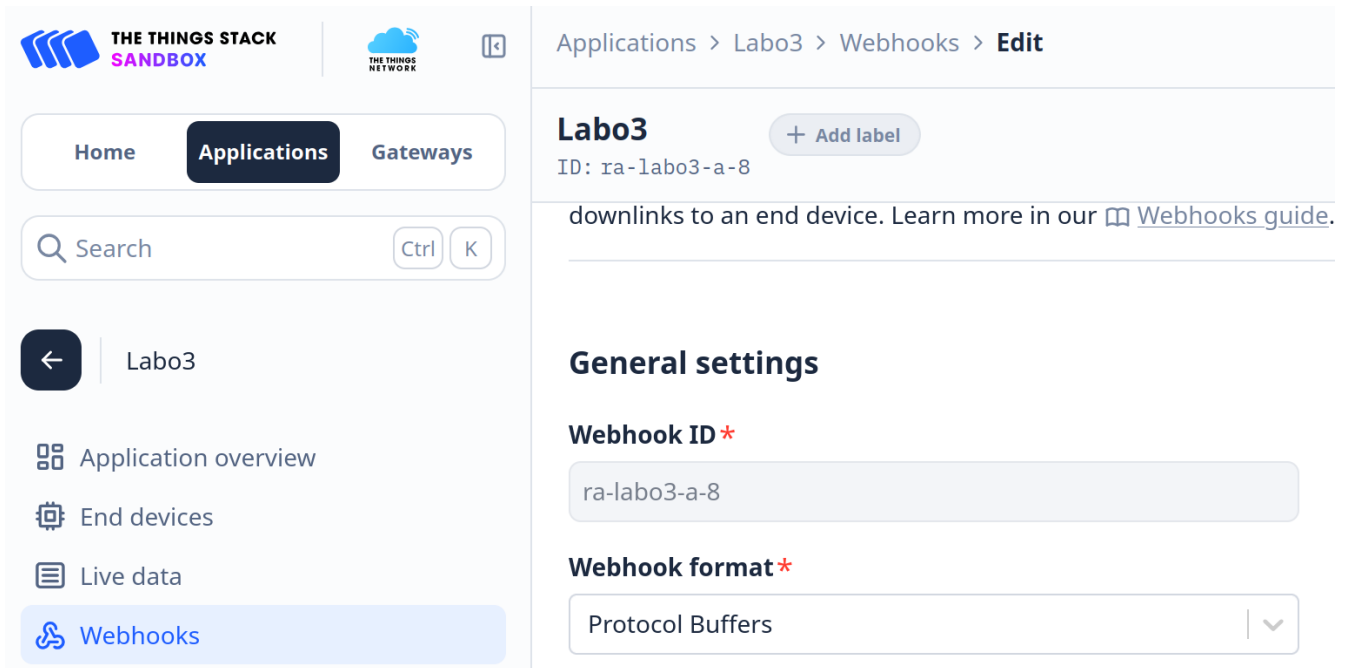


Figure 12: TTN changer le format du Webhook

```

user-8@ds-tinf-ra-sf-8:~/html$ sudo php -S 0.0.0.0:80
[Thu May 7 14:48:09 2026] PHP 8.2.30 Development Server (http://0.0.0.0:80) started
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 Accepted
{"end_device_ids":{"device_id":"ra-lht-65-8","application_ids":{"application_id":"ra-labo3-a-8"},"dev_eui":"A840414881827810",
"join_eui":"4BA4A6DCC4B1C459"},"correlation_ids":["as:up:01KR17P08C3TXZ5N07KCQTFW4A","rpc:/ttn.lorawan.v3.AppAs/SimulateUplink
:2e74614e-5cd9-4e85-963c-de32a627d7a9"],"received_at":"2026-05-07T12:48:19.723557583Z","uplink_message":{"f_port":1,"frm_paylo
ad":"zFQIDQF4AX//f/8=","rx_metadata":{"gateway_ids":{"gateway_id":"test"},"rssi":42,"channel_rssi":42,"snr":4.2},"settings":
{"data_rate":{"lorawan":{"bandwidth":125000,"spreading_factor":7},"frequency":"868000000"},"simulated":true}
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 [200]: POST /script.php
[Thu May 7 14:48:19 2026] 63.34.43.96:41991 Closing
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 Accepted

1

ra-lht-65-8

ra-labo3-a-8@AH00xK00010Y as:up:01KR17QE756EE85EZ9ZPGS8AE4Mrpc:/ttn.lorawan.v3.AppAs/SimulateUplink:6c15118c-ee74-4ba1-8145-c
866e3626babb
000000pp:"
x002 0T
testE(BM(B)ff0@:

00 000
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 [200]: POST /script.php
[Thu May 7 14:48:43 2026] 63.34.43.96:54366 Closing

```

Figure 13: Terninal JSON vs Protocol Buffers

Nous avons mis en place un décodeur permettant de transformer les données brutes en informations exploitables (température, humidité et batterie). Nous avons également utilisé un conteneur pour héberger un serveur recevant ces données via un webhook HTTP.

Les données reçues ont ensuite été traitées par un script PHP, stockées localement, puis affichées sur une interface web avec une mise à jour automatique. Un système d'alertes a également été ajouté afin de signaler les valeurs anormales.

Enfin, nous avons comparé les formats JSON et Protocol Buffers pour la transmission des données. Nous avons constaté que le format JSON est directement exploitable dans notre cas, tandis que le format Protocol Buffers nécessite un traitement supplémentaire car il est binaire.

Annexes

[Payload formatter](#)

[script.php](#)